Microserviço para Ciência de Dados

Julio Cesar Nogueira
2026/2



Julio Cesar Nogueira

Bacharel em Ciência da Computação - Centro Universitário Filadélfia (UniFil) - 2016

Pós Graduação em Internet das Coisas e Inteligência artificial - (Faculdade VINCIT - UNICIV) - 2023

Mestrando - Programa de Pós Graduação em Ciência da Computação - UTFPR - CM

Gerente Especialista - Deloitte - Vale S/A - 2019



Quem é você?

Nome

Idade

Cidade

Trabalho



Calendário 2026/2

Datas e Feriados

- Início: 27/07/2026
- Fim: 04/12/2026
- Pontos de Atenção:
  - Nenhum.

Plano de ensino

Avaliação 1 - Dissertativa Individual
Semana de Congresso de Pesquisa Interinstitucional - 21/10 a 23/10
Avaliação 2 - Objetiva Individual -
Encerramento do Semestre - 26/06
Sistema de avaliação:

- Conhecimentos: 70%
  - Avaliação dissertativa: 30%
  - Avaliação objetiva: 15%
  - Entregas participais ou Atividades Equivalentes: 10%
  - Entrega final de Projetos: 15%

- Habilidades e Atitudes: 30% (Avaliação 360)
- Habilidades:
  - Comunicação Oral e Escrita: 10%
  - Habilidades Cognitivas: 10%
- Atitudes:
  - Autogestão: 2,5%
  - Autonomia: 2,5%
  - Protagonismo: 2,5%
  - Interação: 2,5%

A avaliação 360: 3% Autoavaliação, 3% Equipe (todos os integrantes da equipe avaliam o colega), 24% Professor
MF = (Conhecimentos * 0,70) + (Habilidades e Atitude * 0,30)
NO = (Média Final + Nova Oportunidades) / 2

Avaliação 1 (22/09)

Prova Dissertativa

- Foco: Arquitetura

Avaliação 2 (24/11)

Prova Objetiva

- Foco: Técnico
- Anotações do Spring
- Comandos Docker
- HTTP

Projeto Prático (01/12)

Construção de API

- API em Java
- Documentação Swagger/OpenAPI
- Endpoints REST
- Interoperabilidade

Semana de Congresso de Pesquisa Interinstitucional
21/10 à 23/10

- Palestras
- Seminários
- Apresentações de trabalhos

- Hidden Technical Debt in Machine Learning Systems. (Outras referências)

Hidden Technical Debt in Machine Learning Systems

D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips
{dsculley,gholt,dgg,edavydov,toddphillips}@google.com
Google, Inc.

Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, Dan Dennison
{ebner,vchaudhary,mwyoung,jfcrespo,dennison}@google.com
Google, Inc.

Abstract

Machine learning offers a fantastically powerful toolkit for building useful complex prediction systems quickly. This paper argues it is dangerous to think of these quick wins as coming for free. Using the software engineering framework of *technical debt*, we find it is common to incur massive ongoing maintenance costs in real-world ML systems. We explore several ML-specific risk factors to account for in system design. These include boundary erosion, entanglement, hidden feedback loops, undeclared consumers, data dependencies, configuration issues, changes in the external world, and a variety of system-level anti-patterns.

1 Introduction

As the machine learning (ML) community continues to accumulate years of experience with live systems, a wide-spread and uncomfortable trend has emerged: developing and deploying ML systems is relatively fast and cheap, but maintaining them over time is difficult and expensive.

This dichotomy can be understood through the lens of *technical debt*, a metaphor introduced by Ward Cunningham in 1992 to help reason about the long term costs incurred by moving quickly in software engineering. As with fiscal debt, there are often sound strategic reasons to take on technical debt. Not all debt is bad, but all debt needs to be serviced. Technical debt may be paid down by refactoring code, improving unit tests, deleting dead code, reducing dependencies, tightening APIs, and improving documentation [8]. The goal is *not* to add new functionality, but to enable future improvements, reduce errors, and improve maintainability. Deferring such payments results in compounding costs. Hidden debt is dangerous because it compounds silently.

In this paper, we argue that ML systems have a special capacity for incurring technical debt, because they have all of the maintenance problems of traditional code plus an additional set of ML-specific issues. This debt may be difficult to detect because it exists at the *system* level rather than the code level. Traditional abstractions and boundaries may be subtly corrupted or invalidated by the fact that data influences ML system behavior. Typical methods for paying down code level technical debt are not sufficient to address ML-specific technical debt at the system level.

This paper does not offer novel ML algorithms, but instead seeks to increase the community's awareness of the difficult tradeoffs that must be considered in practice over the long term. We focus on system-level interactions and interfaces as an area where ML technical debt may rapidly accumulate. At a system-level, an ML model may silently erode abstraction boundaries. The tempting re-use or chaining of input signals may unintentionally couple otherwise disjoint systems. ML packages may be treated as black boxes, resulting in large masses of "glue code" or calibration layers that can lock in assumptions. Changes in the external world may influence system behavior in unintended ways. Even monitoring ML system behavior may prove difficult without careful design.

Muro das confusões



O conflito entre Dados e Operações

Dois mundos em choque

- Inovação (Ciência de Dados)
- Estabilidade (Engenharia de Software)

Fluxo de trabalho típico

Do Cientista de Dados ao Engenheiro de Software

- Cientista faz script Python
- Entrega para o Dev

“Isso não escala, não tem tipagem, vai quebrar o servidor”

Desenvolvedor

Projeto morre ou é reescrito do zero



Funciona na minha máquina



Definição de contratos

- Como é feita a definição dos contratos dos microserviços expostos?
- Existe uma documentação?

Nosso Stack de Ferramentas

Ferramentas utilizadas

- Linguagem: Java 17 ou 21 (LTS)
- Framework: Spring Boot
- Documentação: Swagger/OpenAPI
- Infraestrutura: Docker

Onde entra o Python?

Modelos de ML

- Modelos em Python
  - Consumir via API
  - Encapsular em microserviço

Contatos

Email: julio.nogueira@bpkedu.com.br

WhatApp:

